

PRÁCTICA 4

Acceso físico con NFC

Voz · Estrategia

Universidad La Salle México

Aplicaciones Móviles · 900CIB

Equipo GIGN

Aristide Cordonnier · Mathis Gil · Oscar Buat

Docente: M. en C. Felipe Mandujano Gómez

Objetivo y marco teórico

Objetivo

Crear una app con React Native y Expo para usar el teléfono como credencial. La app debe permitir iniciar sesión, mostrar los datos del usuario y enviar su número al lector NFC. El proyecto es del equipo GIGN, para un callcenter de estrategias políticas.

React Native y Expo

React Native permite crear pantallas para el teléfono con JavaScript. Expo ayuda a ejecutar y compilar la app. Usamos Expo Router para pasar entre las pantallas de acceso, credencial y créditos.

Inicio de sesión

Una API recibe una petición de la app y devuelve una respuesta. En esta práctica usamos la API MonsterBallGo. El formulario envía el usuario y la contraseña convertida con SHA-256, como pide el ejemplo del profesor. SHA-256 produce una cadena de texto a partir de la contraseña. La petición viaja por HTTPS. La app no guarda la contraseña (Mandujano Gómez, s. f.-b).

NFC y APDU

NFC permite comunicar dos dispositivos al acercarlos. Android HCE hace que el teléfono responda como una tarjeta. El lector envía un comando APDU y la app devuelve el número de usuario en cuatro bytes. El AID es el código que usa el lector para encontrar nuestro servicio: F012345678 (Google, s. f.).

El número se envía en orden big endian, del byte de mayor peso al de menor peso. Al final se agregan los bytes 90 00. Estos indican que el comando se respondió correctamente. El lector consulta al servidor para decidir si permite el acceso.

El montaje usa un Arduino Nano ESP32 y un lector PN532. En el proyecto del profesor se conectan por HSU, un modo de comunicación serial.

Material y equipo

Recurso	Uso
Computadora con Node.js y JDK	Editar el código y compilar la app
React Native 0.88 y Expo 58	Crear las pantallas
Teléfono Android con NFC y HCE	Instalar la app y probar el lector
Arduino Nano ESP32 y PN532	Leer el número del teléfono
Tres cuentas MonsterBallGo	Probar a cada integrante de GIGN
Servidor y panel del profesor	Consultar y mostrar el resultado del acceso

La demostración usa un teléfono Android compatible con NFC y el lector proporcionado por el profesor.

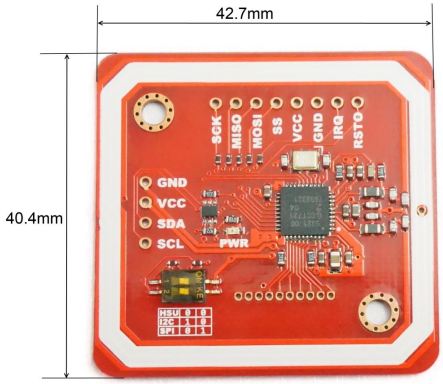


Figura 1. PN532 v3. Imagen del manual de Elechouse, página 2.

El nombre de la app es Voz · Estrategia. En los créditos aparecen los logos de React, Expo, Android, Kotlin, Arduino y Node.js. Los recursos están guardados dentro del proyecto.

Cuentas del equipo

Integrante	Usuario	Número de usuario
Aristide Cordonnier	aristidecordonnier	767108734
Mathis Gil	mathisgil	651791183
Oscar Buat	oscarbuat	877063652

Desarrollo experimental

Organización del proyecto

La app nueva está en expo-client. Se conservaron los directorios app, server y firmware del profesor. Así seguimos usando el mismo lector y servidor.

App.js guarda la sesión. Los tres archivos de src/app muestran el acceso, la credencial y los créditos. src/api.js hace la petición de login. src/nfc.js conecta la pantalla con el servicio NFC escrito en Kotlin.

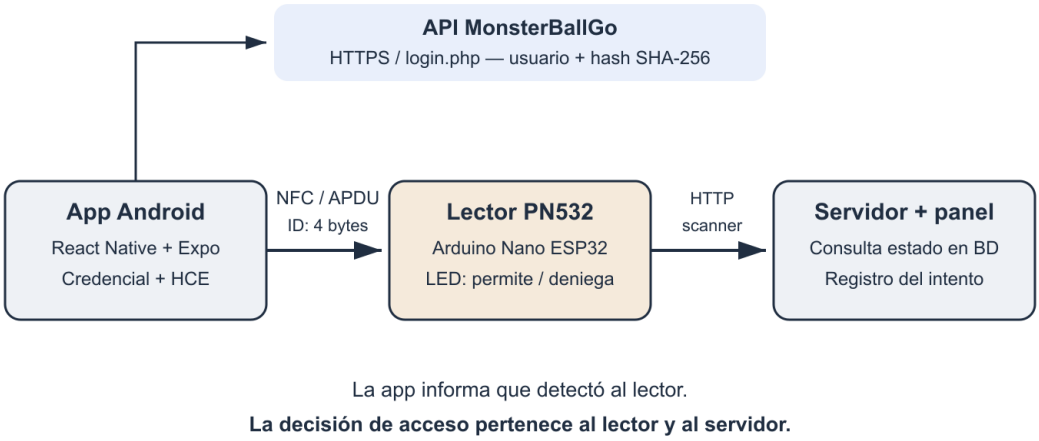


Figura 2. La app inicia sesión con la API. El Arduino consulta al servidor para decidir el acceso.

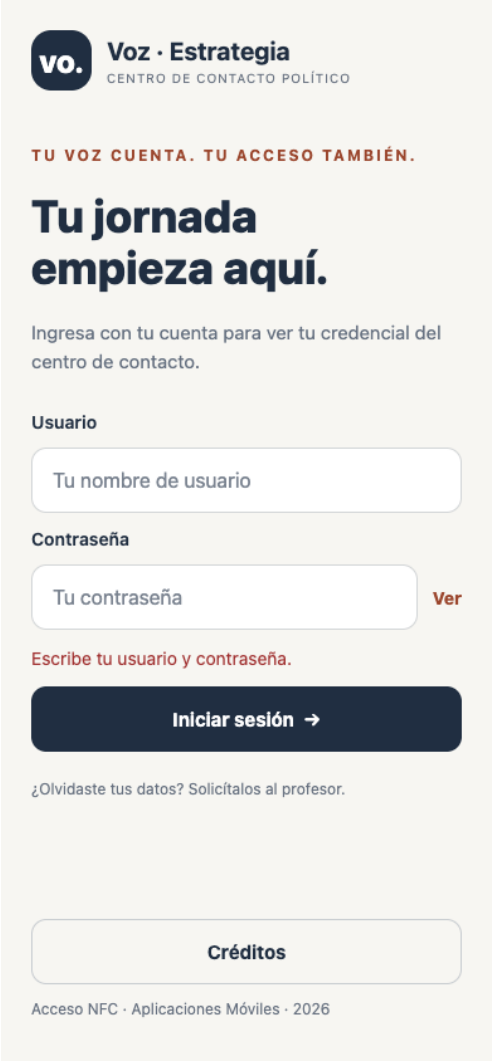
Login y credencial

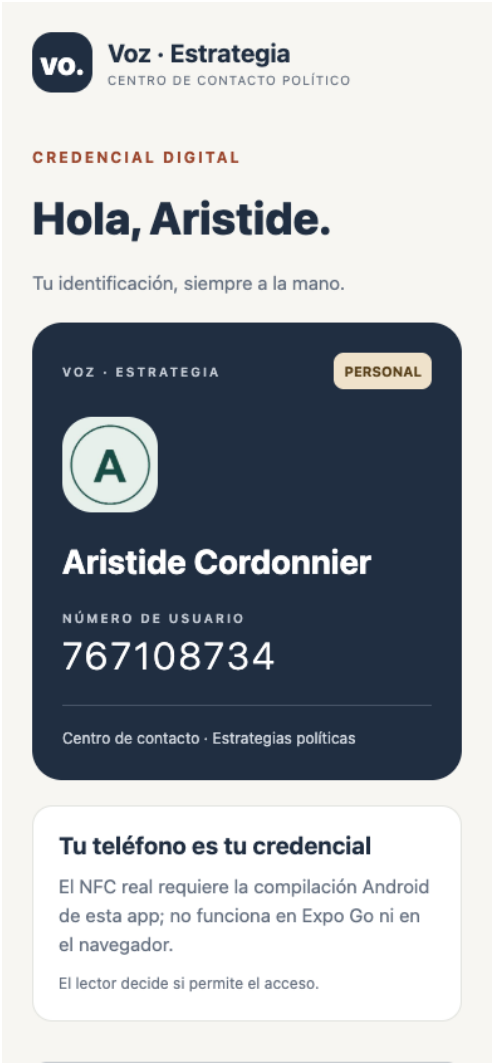
El usuario escribe su cuenta y contraseña. La app comprueba que los dos campos tengan datos y envía el formulario a https://monsterballgo.com/api/login.php. Usa los campos user, pass y token del ejemplo del profesor. Si hay un error, lo muestra en la misma pantalla. La espera máxima es de 15 segundos.

Si la API devuelve un ID válido, la app abre la credencial. Muestra la foto, el nombre completo y el número de usuario. Si la foto no carga, aparecen las iniciales. El botón Créditos abre los nombres del equipo, la fecha y los logos de las tecnologías.

Pantallas de la app

Estas capturas se hicieron en el navegador. Para ver la credencial se cargaron los datos públicos de Aristide en una sesión preparada. Esto permitió revisar el diseño, pero no fue una prueba de login con su contraseña.





Figuras 3 y 4. Formulario de acceso y credencial con foto, nombre completo y número de usuario.

La interfaz se puede revisar en el navegador. Para usar NFC hay que instalar el APK en Android, porque Expo Go no incluye nuestro servicio HCE.

NFC y prueba con el lector

El servicio NFC está en modules/acceso-nfc y tiene dos clases Kotlin. Una recibe el ID desde JavaScript. La otra responde al lector. Al cerrar sesión se borra el ID y se desactiva la respuesta. Si el proceso de la app se reinicia, hay que iniciar sesión otra vez.

El ID debe estar entre 1 y 4294967295 para caber en cuatro bytes. Por ejemplo, 767108734 se envía como 2D B9 26 7E. La respuesta completa es 2D B9 26 7E 90 00. Este ejemplo explica la conversión del código, no una lectura física.

Cuando llega un comando válido, el teléfono vibra y muestra "Lector detectado". El teléfono debe estar desbloqueado. La app no consulta el endpoint del escáner ni muestra si se permitió o se negó el acceso.

Se generó Voz-Estrategia.apk para instalar la app en un teléfono Android con NFC.

Fotos de perfil

Las fotos de Aristide, Mathis y Oscar se cambiaron en la tarea anterior. La consulta de sus perfiles públicos confirmó que conservaban las imágenes personalizadas.

Demostración en clase

Instalar el APK, activar NFC e iniciar sesión. Acercar el teléfono desbloqueado al PN532. Revisar la vibración o el mensaje en la app y mirar el resultado en el panel y en los LED. Cerrar sesión y repetir con cada integrante. El profesor debe bloquear una de las cuentas antes de la prueba.

Caso	Comportamiento esperado
Cuenta habilitada	La app entrega el ID. El lector consulta al servidor y permite el acceso si la cuenta está autorizada.
Cuenta bloqueada	La app entrega el ID de la misma forma. El lector consulta al servidor y niega el acceso.
Sesión cerrada	El servicio deja de entregar el número de usuario.

Para documentar la demostración se necesita una captura del panel web y una foto del Arduino con el LED encendido. Ambas deben corresponder al mismo intento e identificar la cuenta usada. Así se puede comparar lo que recibió el servidor con la señal del montaje.

Análisis de resultados y mejoras

Qué se revisó

Prueba	Resultado
Pantallas en el navegador	Formulario, errores simulados, navegación y cierre de sesión revisados
Compilación Android	APK creado para instalar la app
Login real y lectura NFC	Sin resultados registrados con un dispositivo real

Problemas y soluciones

Expo Go no puede ejecutar nuestro servicio NFC. Por eso se añadió un módulo Kotlin y se compiló un APK propio. Cada cambio en ese módulo requiere volver a compilar e instalar la app.

El login necesitaba el formato exacto de la API. Se usó el ejemplo del profesor para enviar user, pass y token. Esto evita errores por campos incompletos. Las pruebas con respuestas simuladas verifican el manejo de errores, pero no comprueban la contraseña de cada cuenta.

El ID puede superar el límite de un Int de Kotlin. Se guarda como Long y después se divide en cuatro bytes. Si se cortara el número por error, el lector podría recibir el ID de otra persona.

La primera compilación se detuvo al descargar una dependencia. Se repitió con más tiempo de espera y terminó correctamente. Las fotos usan HTTPS y, si fallan, se muestran las iniciales. Así el nombre y el número siguen visibles.

Las pruebas realizadas cubren el software: pantallas, formato de la petición y compilación. Este reporte no incluye resultados de una lectura NFC en un teléfono real. Por eso no se afirma que el montaje haya permitido o negado el acceso. Esa comprobación requiere comparar la cuenta utilizada, el registro del panel y el LED, incluido el caso de la cuenta bloqueada.

Mejoras a futuro

El ID enviado por NFC siempre es el mismo y podría copiarse. Para un uso real haría falta una prueba de identidad que cambie en cada lectura y proteger la conexión del Arduino con el servidor. En el panel sería útil buscar por usuario o fecha y distinguir una cuenta bloqueada de un fallo de red. También conviene ofrecer una tarjeta física a quien no pueda usar el teléfono.

Conclusiones y bibliografía

Conclusiones

A la empresa Voz · Estrategia le proponemos este sistema para controlar la entrada a su callcenter de estrategias políticas. Durante una campaña puede cambiar el personal y haber varios turnos. Con una cuenta por trabajador, la empresa puede retirar el acceso cuando termina un contrato sin esperar a que le devuelvan una tarjeta. El panel también ayuda a revisar un intento de entrada si surge un problema en recepción. Esto resulta útil en un lugar donde se manejan listas de contactos y guiones de llamadas. La app conserva una tarea sencilla: identificar al trabajador. El lector y el servidor toman la decisión de acceso. Para llevar el prototipo a una empresa, primero hay que reforzar la protección del ID y comprobar el funcionamiento del montaje con las cuentas reales.

Desde el punto de vista del trabajador, usar el teléfono como credencial puede ser cómodo, pero no debería ser la única forma de entrar. Una batería descargada, un celular averiado o un modelo sin NFC pueden dejarlo fuera aunque tenga permiso para trabajar. También se le obliga a usar un equipo personal para una necesidad de la empresa. Por eso proponemos mantener una tarjeta de préstamo o un acceso por recepción. El empleado debe saber qué datos se guardan y quién puede ver sus entradas. Además, una lectura de la credencial no demuestra toda su jornada ni debería usarse por sí sola para decidir si trabajó o no.

Bibliografía

Elechouse. (s. f.). PN532 NFC RFID Module User Guide, Version 3.
https://www.elechouse.com/elechouse/images/product/PN532_module_V3/PN532_%20Manual_V3.pdf

Expo. (s. f.-a). Expo SDK 58 reference.
https://docs.expo.dev/versions/v58.0.0/

Expo. (s. f.-b). Add a new module to an existing application.
https://docs.expo.dev/modules/get-started/

Google. (s. f.). Host-based card emulation overview. Android Developers.
https://developer.android.com/develop/connectivity/nfc/hce

Mandujano Gómez, F. (s. f.-a). NFCAccesoFisico [Código fuente]. GitHub.
https://github.com/fmg2099/NFCAccesoFisico

Mandujano Gómez, F. (s. f.-b). expo-login-registro [Código fuente]. GitHub.
https://github.com/fmg2099/expo-login-registro/blob/master/app/index.tsx

Meta. (s. f.). Core Components and APIs. React Native.
https://reactnative.dev/docs/components-and-apis

Devicon contributors. (s. f.). Devicon [Logos]. GitHub.
https://github.com/devicons/devicon